Code Graph Repair: Restoring Silently Disabled Code-Intelligence Tools in the Gemma 4 Developer Agent Corpus

***Abstract*—The Gemma 4 Developer Agent competition ships a 22.4 GB code-intelligence corpus that participant agents use through three tools: `get_code_neighbors`, `get_code_subgraph` and `search_similar_code`. We audited that release and found two defects that degrade it without any error, warning, or log line. First, the dataset describes its per-task graphs and embeddings as hard links to shared commit-level files, but the distributed data keeps only one name per commit: all 129 task-named graph files and 129 task-named embedding files are absent, so 100% of tasks have no graph or embedding file under their own name. Second, the graph builder omits `async def`: 0 of 45,593 AST-extracted async functions are graph nodes. We present *cgr*, a CPU-only, single-file tool whose only dependency is NumPy. It reconstructs every lost file from its commit group and verifies it by SHA-256, and re-indexes each graph with schema-compatible nodes for missing symbols. Because each task's reference patch is known, cgr also supplies the missing evaluation for this corpus: it maps each patch to the symbols it edits and measures whether those symbols are reachable in the graph the agent sees. Applying cgr raises needed-symbol reachability from 90.08% to 100% and async reachability from 0% to 100%, and reduces the tasks fully blind to the asynchronous code their fix edits from 16 to 0. cgr is released under Apache 2.0 as a notebook-embeddable resource that needs no LLM inference and no GPU.**

***Index Terms*—Code Graphs, Software Engineering Agents, Benchmark Integrity, Dataset Repair, Code Retrieval, Reproducibility**

I. Introduction

The Gemma 4 Developer Agent competition [1] provides 129 software-engineering tasks drawn from four Python repositories. An agent must fix a defect in a checkout at a pinned commit, in the style of SWE-bench [3]. Each task ships with a code graph, an embedding index for semantic search, and a reference patch. The graph and the index are how the benchmark's code-intelligence tools see the repository. If they are silently incomplete, the benchmark measures the agent's ability to succeed with a partially deleted toolbelt, unevenly across tasks. This paper documents two such failures, quantifies their effect, and ships a repair.

Defect 1 – lost task-named files. The dataset description states that the 129 task-named files `<instance_id>.json` are hard-linked to commit-named files `<repo>_<base_commit>.json`. In the data as distributed on Kaggle, `graphs/` holds 127 files that form 127 commit groups, and every task-named alias is gone: 129 graph and 129 embedding files are absent. A packaging step that does not preserve hard links produces exactly this. The content survives under the commit name, so nothing is lost, but all 129 tasks find no file under their own name.

Defect 2 – missing asynchronous nodes. Against an independent abstract syntax tree (AST) reference set, the graphs contain 0 of 45,593 async functions, in library code (for example `ahttpx._client.Client.request`) as well as in documentation examples. An extraction rule that matches `FunctionDef` but not `AsyncFunctionDef` produces exactly this, and every tool that traverses the graph inherits the omission.

The evaluation gap. The competition ships no measurement of either defect, and nothing connects the per-task reference patches to the graphs. cgr fills this gap: it turns a packaging complaint into a measurable, per-task quantity.

Contributions. (1) A reproducible audit of the released corpus that quantifies both defects, per task and per repository; (2) cgr, a 1,207-line, single-file tool that reconstructs every lost artifact from its commit group, verifies it by content hash, and re-indexes missing symbols into the released schema; (3) a per-task reachability benchmark built from the reference patches, which measures the fraction of symbols a task's fix edits that the task's own graph contains: 90.08% before repair and 100% after; and (4) a public notebook that runs the whole pipeline on CPU and emits a single `results.json`, from which every number in this paper is generated.

II. Related Work

Representing programs as graphs over syntactic and semantic relations is well established [4]. Repository-level code graphs are now a common substrate for large language model (LLM) software-engineering agents, used to navigate and localise code in RepoGraph [5], CodexGraph [6] and LocAgent [7]. These works build better graphs. cgr asks a prior question: is a graph that has *already shipped* usable? It answers with each task's reference patch as ground truth.

Embedding-based code retrieval, as benchmarked by CodeSearchNet [8], is the model for `search_similar_code`. cgr recovers the embedding files (Defect 1) but does not re-embed, because the released embedder is not part of the dataset (Section VI).

Datasheets [9], Data Cards [10] and the survey of Paullada et al. [11] argue that datasets need documented provenance and known failure modes. cgr is an executable counterpart for a code-intelligence release, where the failure is a filesystem feature that the distribution format silently drops.

Participants independently reported missing graph files and missing async nodes in the competition Discussion **TODO:** link the Discussion threads. Those reports come without a repair, a per-task measurement, or a reusable tool. We reproduced both defects independently; the reports motivate this work but are not its contribution.

III. Method

cgr is a single Python file whose only third-party dependency is NumPy, used to read the `.npz` embeddings. It exposes a library API and a command-line interface, and runs as the four-stage pipeline shown in Fig. 1.

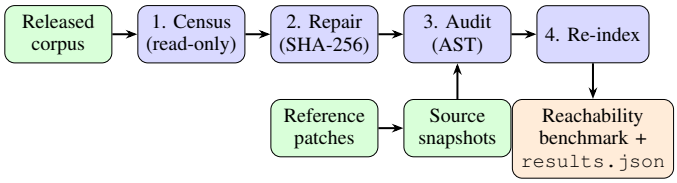


Fig. 1. The cgr pipeline. Census and repair restore the lost per-task files; audit and re-index restore the missing symbols; the reachability benchmark maps each reference patch onto the pre-patch snapshot and is run before and after repair.

A. Census

For each artifact kind (`graphs`, `embeddings`), cgr groups files by (repository, base commit). Every task's own `<instance_id>` file is an expected member of its group. A member is *lost* when it is absent or at most 100 bytes, the size below which the harness disables the code tools. This stage is read-only and runs before anything touches the data.

B. Repair

For each group, cgr takes the surviving member as the true content, copies it to every lost member's path, and verifies each copy by SHA-256. Copying from the survivor rather than re-cloning the repository costs $O(\text{bytes})$ and makes repair a pure function of the release. Groups with no survivor are reported as unrecoverable, never fabricated.

C. Audit

Independently of the release, cgr parses each task's source snapshot with Python's `ast` module and enumerates a reference symbol set R: module-level functions (`def` and `async def`), classes, and the methods defined directly in those classes, excluding test files. With N the set of graph node identifiers, the coverage of symbol kind k is

$$\text{cov}_k = \frac{|R_k \cap N|}{|R_k|}, \tag{1}$$

reported per kind, per repository and per task. Because R and N come from different sources, a disagreement points to the graph builder rather than the data.

D. Re-index

For each graph, cgr adds a node for every symbol in $R \setminus N$. Each node carries the released schema's three fields: an `id` in the dataset's fully qualified `module.Class.symbol` convention, so the graph tools resolve it unchanged, a matching `name`, and the symbol's source `text` at the base commit. cgr does not synthesise vectors or guess edges (Section VI).

E. Reachability Benchmark

Coverage is an intermediate quantity; what matters is whether the symbols a task is *about* are reachable. For each task, cgr maps every removed or replaced line of the reference patch, and every insertion point, onto the pre-patch snapshot the agent sees. The enclosing function, method, or class (for edits in a class body outside any method) is a *needed symbol*; test files are ignored. With S the set of needed symbols over all scored tasks, needed-symbol reachability is

$$\text{reach} = \frac{|S \cap N|}{|S|}. \tag{2}$$

A symbol that is not a node can never be returned by any of the three tools, so this is a hard ceiling on what they can deliver. cgr also reports *affected tasks* (at least one unreachable needed symbol), tasks *fully blind to async* (they need async symbols and their graph contains none of them), *multi-hop connectivity* (the fraction of present needed symbols within two undirected edge hops of another node, which is what `get_code_neighbors` can reach by walking), and a *self-retrieval sanity check* (recall@k when each target's own released vector is the query, which checks the embedding loader and is not a retrieval benchmark).

IV. Experimental Setup

All experiments run on the competition corpus as mounted in a Kaggle Notebook (`gemma-4-developer-agent`: 129 tasks from `Textualize/rich`, `encode/httpx`, `fastapi/fastapi` and `psf/requests`), using the notebook published with this submission. cgr requires Python 3.9+ and NumPy and runs on CPU; no model is loaded. The configuration is the default: a 100-byte threshold and 2-hop connectivity. Every number in Section V is generated from the `results.json` the notebook writes; none was typed by hand.

V. Results

Table I summarises the effect of cgr on the corpus.

A. Defect 1: Lost Task-Named Files

`graphs/` holds 127 commit-named files in 127 groups, and all 129 task-named graph files and 129 task-named embedding files are absent, leaving 100% of tasks without their own files. Repair restores 258 files with 0 unrecoverable groups, i.e. every lost file can be rebuilt from the release itself. Afterwards no graph or embedding file is lost.

TABLE I
EFFECT OF CGR ON THE RELEASED CORPUS

Metric	Before	After
Tasks without their own graph file	129	0
Async node coverage	0.00%	100.00%
Needed-symbol reachability	90.08%	100.00%
async needed symbols	0/44	44/44
Tasks with an unreachable needed symbol	16	0
Tasks fully blind to async	16	0
Needed symbols within 2 hops	79.10%	71.26%

B. Defect 2: Missing Async Nodes

Table II gives node coverage against the AST reference set before re-indexing.

Async coverage is 0% wherever async code exists. The clearest case is `httpx`: sync and class coverage is complete, yet all 98 async functions are missing, which isolates the omission to `async def`. Sync and class coverage is also complete in `rich` and `requests`. The one repository with gaps beyond async is `fastapi`, where many synchronous functions and classes are also absent. We did not attribute that gap to a cause; the benchmark below shows that it lies mostly outside the code the tasks edit. Re-indexing adds 90,653 nodes across 68 tasks (45,593 async, 20,076 sync and 24,984 classes) and raises coverage to 100% for all three kinds.

C. The Cost to Tasks

The reference patches of the 121 scored tasks (8 tasks edit no attributable symbol) edit 494 needed symbols, of which 445 (90.08%) are nodes in the released graphs. By kind, 99.23% of synchronous and 96.72% of class needed symbols are present, against 0 of 44 async ones. The non-async gap therefore barely touches the tasks, while the async gap removes every async symbol a fix needs. Of the 121 tasks, 16 (13.22%) have at least one unreachable needed symbol; these are exactly the 16 tasks that need an async symbol, and all 16 are fully blind to async. After cgr, needed-symbol reachability is 100% (494/494), and affected and fully blind tasks both fall from 16 to 0.

D. Edge Structure

Across the 129 task graphs the release holds 461,336 nodes and 460,018 edges (0.997 per node), all of type `calls`. Of the present needed symbols, 79.10% are within two hops of another node before repair and 71.26% after, because re-indexed nodes carry no edges. The headline numbers measure node presence, but this bounds how much of the repair `get_code_neighbors` can exploit, and synthesising `calls` edges is the natural next step.

E. Sanity Check

Querying the released embeddings with each target's own vector gives recall@1 = recall@10 = 1.00, so the embedding reader is correct where the data survived.

VI. LIMITATIONS

cgr repairs; it does not reconstruct. Repair copies the surviving member of a commit group; a group with no survivor cannot be helped and is reported.

No vectors or edges for new nodes. Re-indexed nodes are found by `get_code_subgraph` and by name, but `search_similar_code` cannot surface them until a host supplies the embedder, which the dataset does not include, and `get_code_neighbors` cannot walk to them. The 100% async coverage is a *node* coverage figure.

The `fastapi` gap is unexplained. We measured it and showed its small effect on tasks, but did not determine whether it reflects a deliberate scope choice by the graph builder or a second bug. Re-indexing fills it regardless, which a host may or may not want.

Needed-symbol extraction is heuristic. It attributes each edited line to the enclosing top-level function, method or class; module-level edits are not attributed. It indicates where an agent must look, not what it must read.

The benchmark is oracle-based. It uses the reference patch, which an agent does not have. It measures the corpus's fitness, not agent behaviour; we ran no agent.

VII. CONCLUSION

As distributed, the Gemma 4 Developer Agent corpus has no task-named graph or embedding file for any of its 129 tasks and represents none of its 45,593 async functions, silently switching off or blinding the code-intelligence tools. cgr restores 258 files and closes the async gap, raising needed-symbol reachability from 90.08% to 100% and cutting fully blind tasks from 16 to 0, on CPU, in one file, with no inference. A code-intelligence release should ship a coverage assertion alongside its artifacts: cgr's audit is under a hundred lines and would have caught both defects before the competition. We release the tool, the notebook and a machine-generated `results.json` so that these claims can be re-derived rather than trusted. This work is original and has not been published elsewhere.

DATA AND CODE AVAILABILITY

This paper is a submission to the Google – The Gemma 4 Developer Agent Paper Track [2] (Best New Resource category). The notebook that produces every number is public at **TODO:** paste the public notebook URL. `cgr.py` is embedded in that notebook and available standalone in the accompanying repository under the Apache 2.0 licence, together with `results.json`.

REFERENCES

[1] E. Markowitz, B. Perozzi, B. Rózemberczki, G. Cameron, H. Hemmati, Y. Li, M. Galkin, M. Farhadi, R. Holbrook, and A. Oldacre, "Google – The Gemma 4 Developer Agent Competition," Kaggle, 2026. [Online]. Available: https://www.kaggle.com/competitions/gemma-4-developer-agent

[2] E. Markowitz, B. Perozzi, B. Rózemberczki, G. Cameron, H. Hemmati, Y. Li, M. Galkin, M. Farhadi, R. Holbrook, and A. Oldacre, "Google – The Gemma 4 Developer Agent Paper Track," Kaggle, 2026. [Online]. Available: https://www.kaggle.com/competitions/gemma-4-developer-agent-paper

TABLE II
GRAPH NODE COVERAGE BY REPOSITORY (FOUND/TOTAL), BEFORE RE-INDEXING

Repository	Tasks	Sync functions	Async functions	Classes
`fastapi`	67	23,448/43,524 (53.87%)	0/45,495 (0.00%)	11,392/36,376 (31.32%)
`rich`	48	43,585/43,585 (100.00%)	–	9,110/9,110 (100.00%)
`requests`	13	3,090/3,090 (100.00%)	–	598/598 (100.00%)
`httpx`	1	520/520 (100.00%)	0/98 (0.00%)	75/75 (100.00%)

[3] C. E. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.
[4] M. Allamanis, M. Brockschmidt, and M. Khademi, "Learning to represent programs with graphs," in *Proc. ICLR*, 2018.
[5] S. Ouyang *et al.*, "RepoGraph: Enhancing AI software engineering with repository-level code graph," in *Proc. ICLR*, 2025.
[6] X. Liu *et al.*, "CodexGraph: Bridging large language models and code repositories via code graph databases," in *Proc. NAACL*, 2025.
[7] Z. Chen *et al.*, "LocAgent: Graph-guided LLM agents for code localization," in *Proc. ACL*, 2025.
[8] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet challenge: Evaluating the state of semantic code search," arXiv:1909.09436, 2019.
[9] T. Gebru *et al.*, "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
[10] M. Pushkarna, A. Zaldivar, and O. Kjartansson, "Data Cards: Purposeful and transparent dataset documentation for responsible AI," in *Proc. ACM FAccT*, 2022.
[11] A. Paullada, I. D. Raji, E. M. Bender, E. Denton, and A. Hanna, "Data and its (dis)contents: A survey of dataset development and use in machine learning research," *Patterns*, vol. 2, no. 11, 2021.